MACHINE LEARNING LABORATORY

Network Intrusion Detection Using Machine Learning

Comparative evaluation of tree classifiers and ensembles on sampled CICIDS2017 network flows

Semester Project Report

Payas Vaishnav
Enrollment number 23BCP430
7th Semester

Computer Science and Engineering
Pandit Deendayal Energy University

Faculty guide
Dr Nandini Modi

22 October 2026

Project outcome

The implemented system compares eight learning configurations, selects Histogram Gradient Boosting using validation macro F1, and provides a local dashboard for flow classification. The measured internal test accuracy is 99.61%, with macro F1 0.9923.

The project includes reproducible training, saved predictions, an executed learning notebook and an interactive demonstration. The evaluation uses a sampled historical benchmark and does not establish live-network reliability.

Abstract and report overview

Network flow records contain measurements that can help distinguish benign communication from malicious traffic. This project develops an offline multiclass intrusion classification system using a sampled CICIDS2017 dataset provided through Western OC2 Lab's IDS-ML repository. The implementation combines a reproducible experiment pipeline with a local web interface for CSV-based inference.

After removing 12,610 duplicate feature rows, the experiment uses 44,051 flows and 77 measurements. Stratified partitions allocate 60% of the distinct flows to training, 20% to validation and 20% to testing. Median imputation fits inside model pipelines. Experiments cover a majority baseline, Decision Tree, a tuned tree, Random Forest, Extra Trees, Histogram Gradient Boosting, a tree using 20 selected features, and a stacking ensemble.

Three-fold training cross-validation supports the comparison, while validation macro F1 determines the selected configuration. The final Histogram Gradient Boosting model achieves 99.61% test accuracy and 0.9923 macro F1. Per-class scores, binary attack-detection errors and stratified bootstrap intervals provide a more useful assessment than accuracy alone.

The contribution is an integrated educational implementation and comparative evaluation. The work credits its source research and data. The test partition was also evaluated during the initial prototype, so the final results remain an internal benchmark. Future work should evaluate on separate capture days or networks using richer collection metadata.

Report structure

Section	Focus
Problem and prior work	Purpose, objectives and relationship to the reference project
Dataset and preparation	Features, class balance and cleaning decisions
Experimental method	Partitions, tuning, model selection and metrics
Models and comparison	Configuration choices and observed tradeoffs
Test evaluation	Confusion matrix, per-class metrics and errors
System and reproduction	Application workflow, rerun commands and limitations

Problem and prior work

Problem statement

A large collection of network flow records is difficult to inspect manually. The task in this project is to learn a mapping from numeric traffic measurements to one of seven known traffic categories, then apply that mapping consistently to new rows with the same feature schema. The output supports investigation of suspicious traffic rather than automatically blocking communication.

Objectives

• Prepare labeled flow data and prevent identical feature rows from crossing partition boundaries.

• Compare simple and ensemble classifiers with a majority baseline and class-sensitive metrics.

• Study whether hyperparameter tuning and reducing the feature set improve validation performance.

• Save the complete fitted preprocessing and classification pipeline for repeatable inference.

• Provide a clear demonstration with downloadable predictions and an inspectable error analysis.

Relationship to the source research

Western OC2 Lab’s IDS-ML repository contains notebook implementations associated with tree-based IDS, MTH-IDS and LCCDE research. Yang et al. describe a tree-based intrusion detection approach in their GLOBECOM 2019 paper [1]. This project adopts that general application and uses the repository’s sampled CICIDS2017 CSV [2].

The submitted code adds a separate reproducible benchmark and dashboard. It compares scikit-learn models, introduces a baseline, documents partitioning, runs a bounded tree hyperparameter search, and reports errors. It does not claim to reproduce the complete MTH-IDS or LCCDE frameworks, and Histogram Gradient Boosting is not presented as XGBoost.

Scope of the application

Input consists of numeric flow measurements, not packet payloads or raw PCAP files. The application predicts known labels from an uploaded CSV. Live packet capture, traffic blocking, alert delivery and unknown-attack detection are outside the implemented scope. These boundaries keep the demonstration reproducible on an ordinary student laptop.

Dataset and preparation

CICIDS2017 contains benign traffic and multiple attack scenarios with flow measurements derived using CICFlowMeter [3]. This project uses the upstream sample file rather than the complete original dataset. The labels in this sample are BENIGN, Bot, BruteForce, DoS, Infiltration, PortScan and WebAttack.

Data property	Value
Source sample rows	56,661
Removed duplicate feature rows	12,610
Retained distinct flows	44,051
Numeric feature columns	77
Traffic labels	7

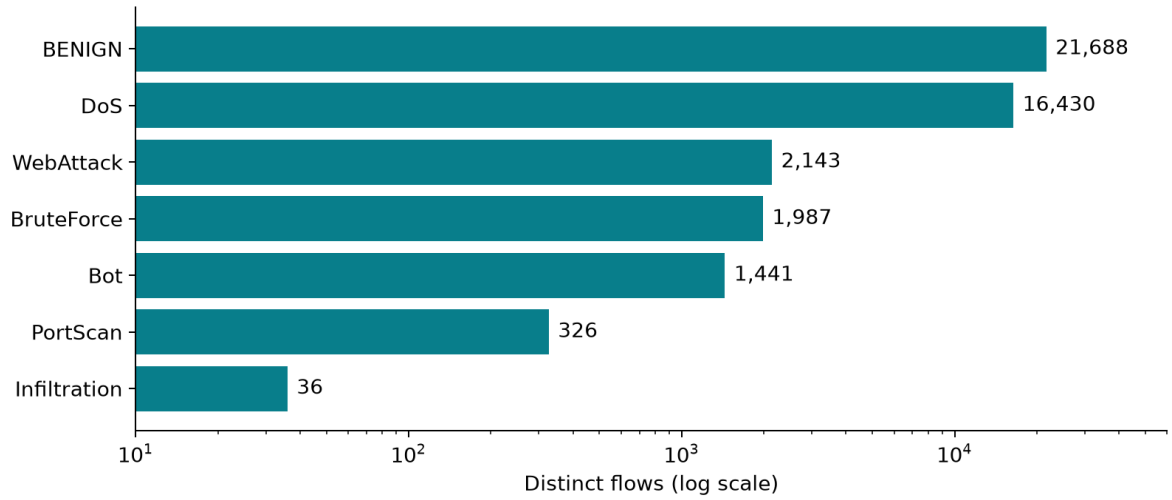


Figure 1 Retained class counts on a logarithmic horizontal axis

The rarest label, Infiltration, contains only 36 distinct flows. A perfect result on its small test subset is therefore weak evidence. Macro F1 gives each category equal influence, and class-balanced fitting increases the influence of scarce labels where supported by the classifier.

Preparation trims column names and labels, removes missing labels and exported index columns, converts feature values to numeric form, and treats infinite values as missing. Deduplication keeps the first row for identical feature vectors, including conflicting-label duplicates. Median imputation then fits on each fitting partition. The script retains constant features; univariate feature selection can issue warnings for them, which the saved training log records.

Experimental method

Partition	Distinct flows	Use
Training	26,430	Fit models and perform tree search
Validation	8,810	Select configuration by macro F1
Test	8,811	Report selected model performance

Training and selection procedure

1. Deduplicate flow features and create stratified partitions with random seed 42.

2. Search 12 tree settings with three-fold stratified cross-validation on the training partition only.

3. Evaluate all eight configurations with three training folds, then fit each on the complete training partition and predict validation labels.

4. Select the non-baseline configuration with the highest validation macro F1.

5. Refit that configuration on training plus validation data, then predict the fixed test partition.

6. Save metrics, the fitted pipeline, partition identifiers, error rows and example inputs.

Metrics

Accuracy is the number of correctly classified flows divided by the total number of flows. For a class, precision is TP / (TP + FP), recall is TP / (TP + FN), and F1 is 2 × precision × recall / (precision + recall). Macro F1 averages class F1 values equally. A confusion matrix identifies which actual categories the model confuses.

For operational interpretation, labels also map to normal versus attack. The false alarm rate is benign flows predicted as attack divided by all benign flows. Attack recall is detected attack flows divided by all attack flows.

Evaluation qualifications

The fixed sample test split also appears in the initial prototype evaluation. The expanded benchmark keeps selection on validation but does not create an independent external test. Correlated flows may remain across a random split. The tuned tree’s repeated CV score reuses search folds, so it is a diagnostic rather than a nested-CV generalization estimate. Bootstrap intervals reflect resampling of these test examples, not collection-level independence.

Models and configuration choices

Configuration	Main settings
Majority baseline	Always predicts the most frequent training label
Decision Tree	Maximum depth 20, balanced class weights
Tuned Decision Tree	12 random candidates across depth, leaf size and split criterion
Random Forest	120 trees, balanced class weights
Extra Trees	120 trees, balanced class weights
Histogram Gradient Boosting	120 maximum iterations, 15 leaves, balanced class weights
Top 20 features Tree	Training-only ANOVA ranking, 20 features, depth 20
Stacking ensemble	Decision Tree and 60-tree Extra Trees; logistic regression meta-model; 3 folds

The Decision Tree learns feature thresholds. Random Forest averages randomized trees, and Extra Trees adds randomness in threshold construction. Histogram Gradient Boosting builds trees sequentially to improve the current model. Stacking fits a meta-classifier on cross-validated base-model predictions [4]. These methods explore different ways to use nonlinear relationships in tabular measurements.

Hyperparameter search

The tree search considers maximum depth in {8, 12, 20, 30, unlimited}, minimum leaf size in {1, 2, 4, 8}, and criterion in {Gini, entropy}. It samples 12 of the 40 combinations using seed 42 and chooses by training CV macro F1. The selected settings are recorded below.

min_samples_leaf = 1

max_depth = None

criterion = gini

Feature selection provides a separate ablation: can 20 measurements retain enough information for accurate classification? The selector fits only within the fitting pipeline. It reduces the inputs to the tree, but the dashboard still requires the common full input schema for consistent validation.

Validation and cross validation results

Configuration	Val accuracy	Val macro F1	CV mean ± SD
Majority baseline	49.24%	0.0943	0.0943 ± 0.0000
Decision Tree	99.35%	0.9641	0.9076 ± 0.0512
Tuned Decision Tree	99.27%	0.9546	0.9081 ± 0.0511
Random Forest	98.94%	0.9277	0.9166 ± 0.0460
Extra Trees	98.35%	0.9481	0.8958 ± 0.0365
Histogram Gradient Boosting	99.33%	0.9653	0.9296 ± 0.0404
Top 20 features Tree	98.42%	0.9143	0.8948 ± 0.0550
Stacking ensemble	98.41%	0.8503	0.8478 ± 0.0035

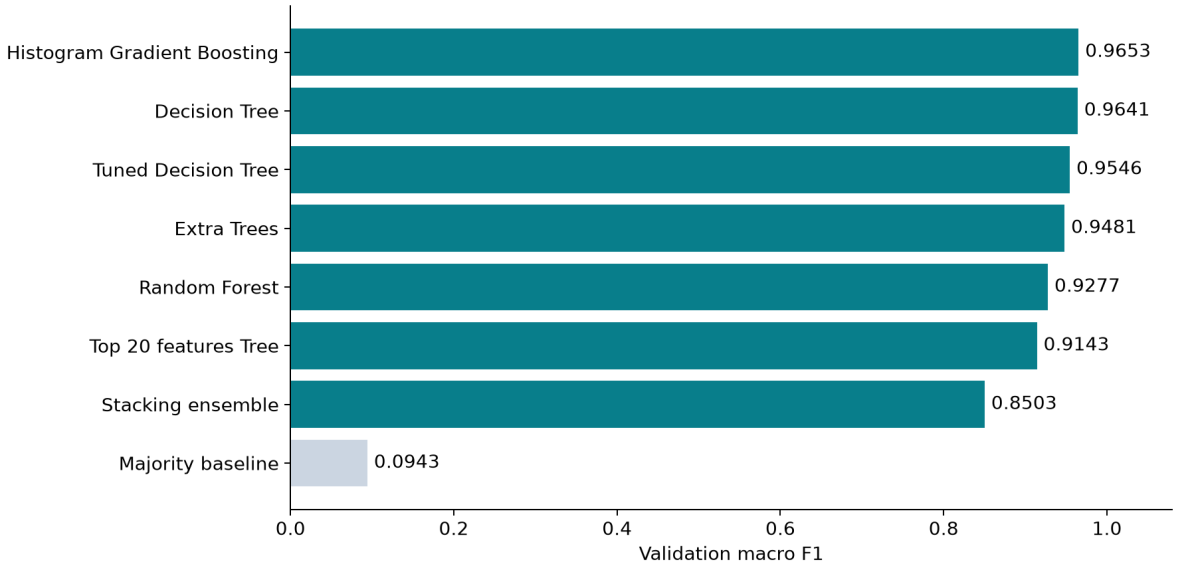


Figure 2 Validation macro F1 for all eight configurations

Histogram Gradient Boosting narrowly exceeds the original Decision Tree on validation macro F1. Stacking and the 20-feature tree score lower in this experiment. The majority baseline achieves about 49% validation accuracy while its macro F1 is only about 0.0943, showing why accuracy alone is insufficient. The observed CV variation also cautions against treating small score differences as conclusive.

Test performance and confusion matrix

Selected model	Test accuracy	Test macro F1
Histogram Gradient Boosting	99.61%	0.9923

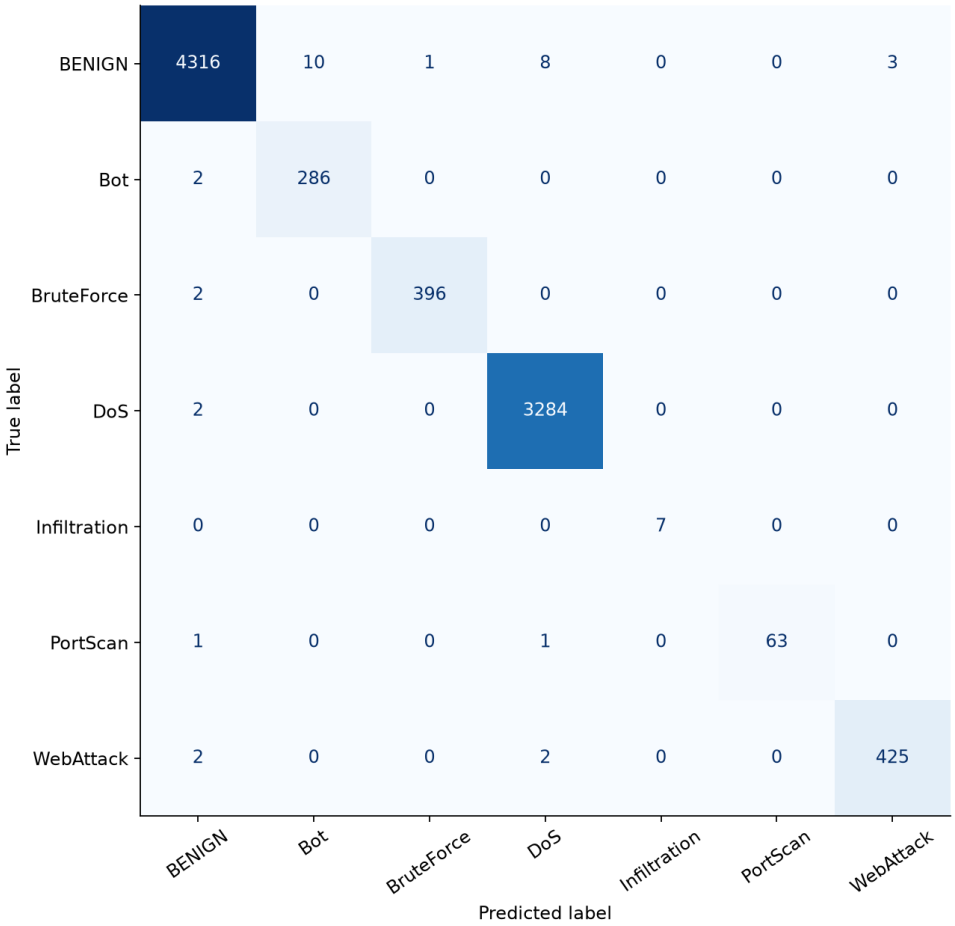


Figure 3 Test confusion matrix with actual labels on rows and predictions on columns

The selected model makes 34 multiclass mistakes among 8,811 test examples. Most observations lie on the diagonal. Off-diagonal cells show both benign-versus-attack errors and confusions between attack categories.

A 300-resample stratified bootstrap gives an accuracy interval of 99.48% to 99.74% and a macro F1 interval of 0.9882 to 0.9955. These are descriptive 95% percentile intervals for this sample, without guarantees under dataset shift.

Per class results and missed attacks

Traffic label	Precision	Recall	F1	Support
BENIGN	0.9979	0.9949	0.9964	4338
Bot	0.9662	0.9931	0.9795	288
BruteForce	0.9975	0.9950	0.9962	398
DoS	0.9967	0.9994	0.9980	3286
Infiltration	1.0000	1.0000	1.0000	7
PortScan	1.0000	0.9692	0.9844	65
WebAttack	0.9930	0.9907	0.9918	429

Actual traffic	Predicted normal	Predicted attack
Normal	4316	22
Attack	9	4464

The model flags 22 benign flows as attacks and misses 9 attack flows by predicting BENIGN. Attack recall is 99.80%; the benign false alarm rate is 0.51%. Other multiclass errors confuse attack categories without missing their attack status.

A false alarm can increase investigation workload. A missed attack can allow malicious activity to pass unnoticed. This implementation uses the classifier’s standard predicted label; it does not optimize a cost-sensitive alert threshold. Its confidence values are not calibrated risk probabilities.

Bot has the lowest per-class test F1 in this run. Infiltration has only seven test examples, so its score cannot establish reliable detection of that class. The CSV in artifacts/error_examples.csv retains feature measurements with actual and predicted labels for inspection.

Practical interpretation

The measured scores support the feasibility of classification within this sampled benchmark. They do not prove the system will detect a novel attack or remain accurate when traffic characteristics change. Independent evaluation by capture day, device or network is the most useful next validation step.

Feature analysis and application design

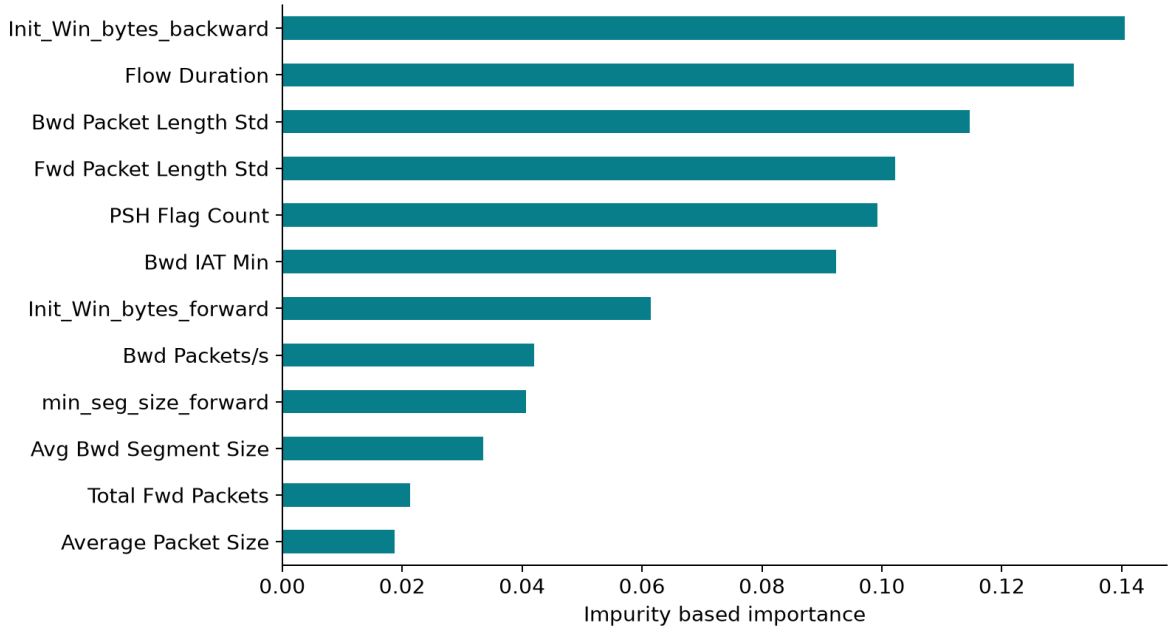


Figure 4 Diagnostic Decision Tree feature importance for interpretation

Histogram Gradient Boosting does not expose impurity-based feature importance through the interface used here. The chart therefore comes from a separate diagnostic Decision Tree fitted on the development partition. It is explicitly labeled as a diagnostic model. It does not explain the selected boosting model or establish causal importance.

Application components

Module	Responsibility
ids.py	Validate CSVs, clean numeric values and run saved-model inference
train.py	Tune, compare, select, evaluate and save artifacts
app.py	Display results, experiments and downloadable CSV predictions
verify.py	Check inference, schema rejection and dashboard demo interaction
IDS_Experiment.ipynb	Executed walkthrough of dataset, results and predictions

The dashboard provides Model results, Detect attacks, Experiments and Understand the project tabs. Inference loads the locally generated pipeline, requires matching feature columns, limits uploads to 50,000 rows, and exports predictions with labels and confidence. Uploaded files are processed in memory; the application does not add them to the training set.

Reproduction and demonstration

Reproduce the experiment

1. Install the pinned packages from requirements-lock.txt in a Python 3.13 virtual environment. The verified environment uses scikit-learn 1.9.1.

2. Ensure reference/data/CICIDS2017_sample.csv is present. setup.ps1 can download the upstream repository.

3. Run .venv\Scripts\python.exe train.py, then .venv\Scripts\python.exe verify.py.

4. Run .venv\Scripts\python.exe prepare_materials.py to refresh the measured results guide and executed notebook.

5. Start the dashboard with .venv\Scripts\python.exe -m streamlit run app.py. Restart after retraining to clear the model cache.

Evidence retained

metrics.json records final scores, bootstrap intervals, versions, seed and data checksum. comparison.csv stores validation and CV results. tuning.csv records all 12 search candidates. partitions.csv provides deterministic row-to-partition assignments. test_predictions.csv and error_examples.csv allow independent metric and error checks. The training log records execution output and feature-selection warnings.

Dataset SHA-256

03ba3626a0f9bb73b90c56772893e68b0577285130891cd7d811542c17b032dd

Demonstration sequence

• Explain the traffic classification problem and the seven labels.

• Show the baseline comparison, validation selection and test confusion matrix.

• Open Detect attacks, retain demo flows and click Analyze flows. These are held-out test rows.

• Download the predictions, then show error groups in the Experiments tab.

• Explain one limitation and the next independent evaluation you would perform.

The supplied demo flows are held-out examples with target labels removed from the input. After setup, local model inference works without internet access. Keep the trained artifacts and pinned environment available on the demonstration computer.

Conclusions and references

Conclusions

The project delivers a working offline intrusion classification system with a reproducible benchmark and a usable interface. Among eight evaluated configurations, Histogram Gradient Boosting achieves the highest validation macro F1. The selected model reaches 99.61% accuracy and 0.9923 macro F1 on the fixed internal test split.

The experiments also reveal useful negative results: tuning does not automatically improve validation performance, a small feature subset loses information, and the tested stacking configuration underperforms the simpler tree. Those findings support choosing a model from evidence rather than complexity alone.

Limitations and future work

The sample is historical, rare classes are small, exact deduplication cannot remove all correlated observations, and the test split has been used previously. Bootstrap intervals do not model these effects. Future work should use full capture metadata to evaluate by day or session, test a separate deployment network, calibrate confidence values on separate data, and measure live extraction and inference latency before integrating alerts.

References

[1] L. Yang, A. Moubayed, I. Hamieh and A. Shami. Tree-Based Intelligent Intrusion Detection System in Internet of Vehicles. IEEE GLOBECOM, 2019. DOI 10.1109/GLOBECOM38437.2019.9013892.

[2] Western OC2 Lab. Intrusion Detection System Using Machine Learning. https://github.com/Western-OC2-Lab/Intrusion-Detection-System-Using-Machine-Learning. Sample and reference revision 502ba86173a8aae5a9b2973fa4e0a42de69fa6ba. Upstream MIT license retained in reference/LICENSE.

[3] Canadian Institute for Cybersecurity, University of New Brunswick. CICIDS2017 dataset documentation. https://www.unb.ca/cic/datasets/ids-2017.html. Accessed 22 October 2026.

[4] scikit-learn documentation. Ensemble methods. https://scikit-learn.org/stable/modules/ensemble.html. Accessed 22 October 2026.

[5] scikit-learn documentation. Cross-validation. https://scikit-learn.org/stable/modules/cross_validation.html. Accessed 22 October 2026.